Block 5 – SDN & Cloud-Native Networking

SDN, NFV, Overlays & Network Automation

Arquitectura de Xarxes

Universitat Pompeu Fabra

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. Cloud-Native Applications
5. From VLANs to Overlay Networks
6. Container Networking
7. Infrastructure as Code & Network Automation
8. Session Summary

upf.

Outline

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. Cloud-Native Applications
5. From VLANs to Overlay Networks
6. Container Networking
7. Infrastructure as Code & Network Automation
8. Session Summary

upf.

What If You Could Program Your Entire Network?

How are modern networks built and automated under the hood?

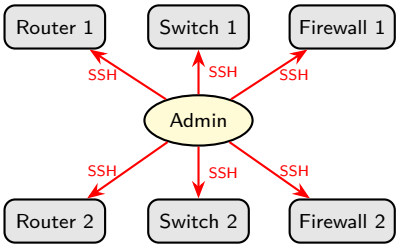
Traditional networks are configured device by device.
What if the network was as programmable as software?

Learning objectives:

1. Explain SDN architecture and why centralized control matters
2. Understand NFV and how it replaces hardware appliances
3. Describe overlay networks (VXLAN) and why they scale beyond VLANs
4. Explain how containers communicate and how Infrastructure as Code automates network management

Traditional Network Management

- Each device (router, switch, firewall) configured **individually**
- Configuration via Command-Line Interface (CLI), **device by device**
- Vendor-specific commands and interfaces
- Changes require **manual intervention** on every device



Each device: separate login, separate config

Imagine updating 1,000 switches one by one. That is traditional networking.

Scalability Challenges

- Data centers grow from hundreds to **hundreds of thousands** of devices
- Manual configuration does **not scale**
- Network changes are **slow**: days or weeks for approval + implementation
- **Human errors** increase with complexity
- Cloud-scale infrastructure demands **automation**

The Control Plane Problem

- In traditional networks, **every device** runs its own control plane
- Each router independently computes routes (OSPF, BGP from Block 3)
- No **centralized view** of the entire network
- Difficult to implement **network-wide policies**
- Limited **programmability**: cannot easily add new features

The control plane is **distributed** by design. What if we **centralized** it?

upf.

Discussion: The Cost of Manual Networks

Why have traditional networks survived so long despite their scalability problems?

- Hint: think about reliability, vendor relationships, and risk aversion
- What would it take for an organization to change?

Outline

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. Cloud-Native Applications
5. From VLANs to Overlay Networks
6. Container Networking
7. Infrastructure as Code & Network Automation
8. Session Summary

upf.

Why SDN?

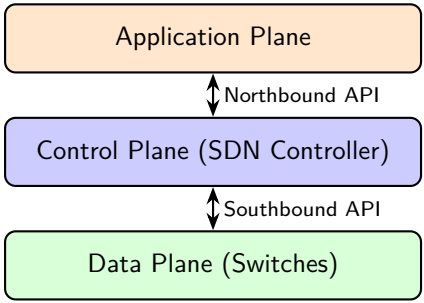
Traditional networks have no central brain: every device runs its own control logic, independently. Updating a policy means logging into each device separately.

The key insight: what if we **separated the decision-making** (control plane) from the **packet forwarding** (data plane) and moved all decisions into a single piece of software?

That is the core idea behind Software-Defined Networking (SDN).

SDN: Core Idea

- **Separate** the control plane from the data plane [1]
- **Centralize** network intelligence in a software controller
- **Program** network behavior through APIs



[1] McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM CCR*, 2008.

SDN Architecture: Three Planes

Data Plane (Infrastructure Layer):

- Physical/virtual switches that **forward packets**
- No longer make independent routing decisions
- Receive forwarding rules from the controller

Control Plane (Controller):

- Centralized software with a **global view** of the network
- Computes paths, installs forwarding rules on switches
- Single point of management for the entire network

Application Plane:

- Applications that define **network behavior** (firewall, load balancer, monitor)
- Communicate with the controller via northbound APIs

upf.

SDN Controllers

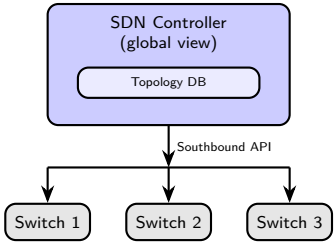
- The controller is the **brain** of the network: it has a global view and makes all forwarding decisions
- Maintains a real-time **topology database**
- Supports **high availability** via clustering

Open-source examples:

- **OpenDaylight** (Linux Foundation)
- **ONOS** (Open Network Operating System, carrier-grade)

Proprietary examples:

- **Cisco ACI**, **VMware NSX**



Flow rules pushed to all devices at once

Southbound Interface: OpenFlow

- **OpenFlow** (2008, Stanford): the first standard southbound protocol
- Controller pushes **flow rules** to switches: match on packet fields, apply an action (forward, drop, redirect)
- Proved the concept: a controller can program any switch centrally
- Rarely used in production today; modern SDN relies on **gNMI** (gRPC Network Management Interface), **NETCONF** (Network Configuration Protocol) / **YANG** (Yet Another Next Generation data modelling language), and vendor APIs (Cisco ACI, VMware NSX) [1]

[1] ONF, "Software-Defined Networking: The New Norm for Networks," ONF White Paper, 2012.

Northbound Interface: Representational State Transfer (REST) APIs

- Applications interact with the controller via **REST APIs**
- Example operations:
  - `GET /topology/links` → get network topology
  - `POST /flows` → add a flow rule
  - `GET /statistics/port` → query traffic statistics

Key benefit: the network becomes **programmable** like any other software system; any developer can write applications that control network behavior.

SDN Benefits: Summary

- **Centralized management**: one controller, global network view
- **Programmability**: automate changes via APIs
- **Agility**: deploy new policies in seconds, not days
- **Vendor independence**: open protocols (gNMI, NETCONF) and standard APIs reduce lock-in
- **Innovation**: researchers and developers can experiment easily

SDN is the foundation for modern cloud networking: AWS, Azure, and GCP all use SDN internally.

upf.

Discussion: SDN Trade-offs

If the SDN controller fails,
what happens to the network?

- Hint: single point of failure vs distributed control
- How do real deployments address this? (clustering, failover)

Outline

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. **Network Function Virtualization (NFV)**
4. Cloud-Native Applications
5. From VLANs to Overlay Networks
6. Container Networking
7. Infrastructure as Code & Network Automation
8. Session Summary

upf.

Why NFV?

SDN solves *how* traffic flows: a central controller programs forwarding rules across the network. But the functions that actually **process** that traffic (firewalls, load balancers, routers) are still dedicated hardware boxes: expensive, slow to procure, and impossible to spin up in seconds.

SDN can steer traffic perfectly, but if it leads to a €50K hardware appliance, we have not solved the flexibility problem.

NFV completes the picture: run those functions as **software on the same commodity servers** SDN already manages.

NFV: From Hardware to Software

Traditional network functions run on **dedicated hardware boxes**:

- Firewall: €10K–€100K+
- Load balancer, router, IDS/IPS

Problems: **expensive**, inflexible, slow to deploy, vendor lock-in

NFV replaces them with **software (VNFs)** running on standard x86 servers or VMs. [1]

Physical Appliance	VNF Equivalent
Hardware firewall	pfSense, iptables
Hardware load balancer	HAProxy, NGINX
Hardware router	VyOS, FRRouting
IDS/IPS	Snort, Suricata

[1] ETSI GS NFV 002, "NFV Architectural Framework," v1.2.1, 2014.

upf.

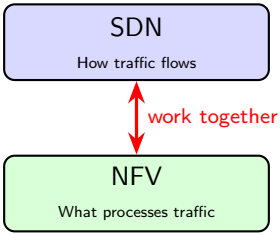
NFV + Cloud Integration

- Deploy VNFs on cloud infrastructure (IaaS) $\rightarrow$ **minutes** instead of months
- **Scale horizontally**: add more instances under load
- **Update easily**: software upgrades, no truck rolls
- **Cost reduction**: commodity servers vs specialized hardware

No hardware procurement, no shipping, no rack-and-stack.

upf.

SDN + NFV: Complementary Technologies



Example: HTTP request entering a data center

1. SDN controller sees incoming traffic
2. Forwards it to a **VNF firewall** (NFV) for inspection
3. Firewall approves; SDN routes it to the **VNF load balancer**
4. Load balancer distributes to backend servers

- SDN **steers** traffic (forwarding rules)
- NFV **processes** traffic (filter, balance, inspect)

Discussion: When Hardware Still Wins

Can you think of a scenario where a physical appliance is better than a VNF?

- Hint: think about latency, throughput, and specialized workloads
- What about hardware accelerators (FPGAs, SmartNICs)?

Outline

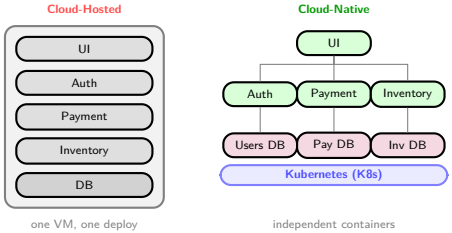
1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. **Cloud-Native Applications**
5. From VLANs to Overlay Networks
6. Container Networking
7. Infrastructure as Code & Network Automation
8. Session Summary

upf.

What Does Cloud-Native Mean?

SDN and NFV give us programmable, automated infrastructure. But the applications running on top can still be designed the old way: one big monolith, manually deployed, that happens to run in the cloud. **Cloud-native** is the design philosophy that fully exploits what SDN, NFV, and containers make possible.

	Cloud-Hosted	**Cloud-Native**
Design	Monolith in a VM	Microservices
Scaling	Manual / vertical	Auto / horizontal
Failure	Restart whole app	Isolate service
Deploy	Manual steps	CI/CD pipeline



[1] CNCF, "Cloud Native Definition v1.0," github.com/cncf/toc, 2018.

Cloud-Native Design Principles

Cloud-native applications are built around a small set of principles: [1]

- **Microservices**: one responsibility per service; independent deploy and scale
- **Containers**: standard, portable packaging; same image from laptop to production
- **Declarative configuration**: describe *what* you want, not *how* to achieve it
- **Immutable infrastructure**: never patch a running instance; replace it with a new image
- **Automated delivery**: CI/CD pipelines build, test, and deploy without manual steps

[1] Burns et al., *Kubernetes: Up and Running*, 3rd ed., O'Reilly, 2022.

upf.

Kubernetes: The Cloud-Native Platform

These principles require a platform that handles scheduling, networking, scaling, and self-healing automatically. That platform is **Kubernetes (K8s)**, introduced in Block 4 Session 2.

From a networking perspective, K8s provides:

- Each **Pod** gets its own IP address (dynamic, managed by the platform)
- **Services** provide stable DNS names regardless of which host Pods run on
- **Ingress** controllers expose services externally with L7 routing
- **Network Policies** define which Pods can talk to which (micro-segmentation)
- **Container Network Interface (CNI)** plugins handle the overlay wiring (typically VXLAN-based)

Cloud-native networking requirements (dynamic IPs, cross-host comms, load balancing, policies) are exactly what K8s was built to solve.

upf.

Discussion: Is Cloud-Native Always Better?

A bank has a 15-year-old monolithic payment application running on two dedicated servers.
Should they rewrite it as cloud-native microservices?

- Hint: think about cost, risk, team skills, and whether the current system actually has a scaling problem
- Is "cloud-native" a tool for specific problems, or always the right answer?

Outline

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. Cloud-Native Applications
5. **From VLANs to Overlay Networks**
6. Container Networking
7. Infrastructure as Code & Network Automation
8. Session Summary

upf.

Why Overlay Networks?

SDN and NFV give us programmable, virtualized infrastructure. Many tenants share the same physical network, so we need to **isolate their traffic**. We already know VLANs (Block 3), but VLANs are capped at 4,096 networks and cannot span Layer 3 boundaries.

A cloud provider hosting millions of tenants, each running SDN-controlled VNFs, cannot rely on VLANs. A scalable isolation mechanism is needed: one that works across any physical topology and is invisible to the underlay.

That is what overlay networks solve.

The VLAN Scalability Problem

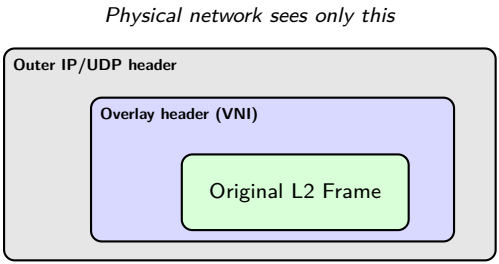
- In Block 4 we used **VLANs** to isolate tenants on virtual networks
- VLANs use a **12-bit ID** $\rightarrow$ maximum **4,096** virtual networks
- Cloud providers host **millions** of tenants $\rightarrow$ VLANs are not enough
- Additional limitations:
  - VLANs are confined to a **single L2 domain**
  - Moving a VM to another host may require VLAN reconfiguration

We need a technology that scales beyond 4K networks and works across L3 boundaries.

upf.

Overlay Networks: Concept

- An **overlay network** is a virtual network built **on top of** the physical one
- Uses **encapsulation**: the original frame is wrapped inside a physical packet
- The physical network only sees the **outer headers**: tenant traffic is invisible
- Each tenant gets **full isolation** without the physical network knowing about them



VXLAN: Overview

- **VXLAN** (Virtual eXtensible LAN): most widely used overlay protocol
- Encapsulates L2 frames in **UDP packets** over the physical network
- Uses a **24-bit VNI** (VXLAN Network Identifier)
  - $2^{24} =$ **16 million** virtual networks (vs 4,096 VLANs)
- Endpoints: **VTEPs** (VXLAN Tunnel Endpoints) [1]
  - Encapsulate at source, decapsulate at destination

[1] IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.

VXLAN: How It Works

- VM A sends a frame → VTEP 1 wraps it in UDP with a VNI
- Travels over the physical network as a regular UDP packet
- VTEP 2 strips outer headers, delivers original frame to VM B
- VMs on the **same VNI** form an isolated L2 domain

Overlay Benefits for Scalability

- **16 million virtual networks** (vs 4,096 VLANs)
- Physical network does not need to know about tenants
- VMs can **migrate** across hosts without reconfiguring the underlay
- Decouples **virtual topology** from physical topology [1]

[1] IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.

Discussion: Overlay Networks

Why can the physical network remain simple
if we use overlay networks?
What are the trade-offs of encapsulation?

Hints: think about overhead (extra headers), Maximum Transmission Unit (MTU) implications, and troubleshooting complexity.

Outline

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. Cloud-Native Applications
5. From VLANs to Overlay Networks
6. **Container Networking**
7. Infrastructure as Code & Network Automation
8. Session Summary

upf.

Why Container Networking?

Overlays give us scalable, isolated virtual networks across any physical infrastructure. Now the question is: what runs *inside* those networks? Modern applications are no longer monolithic; they are split into dozens of small, independent services, each deployed as a **container**.

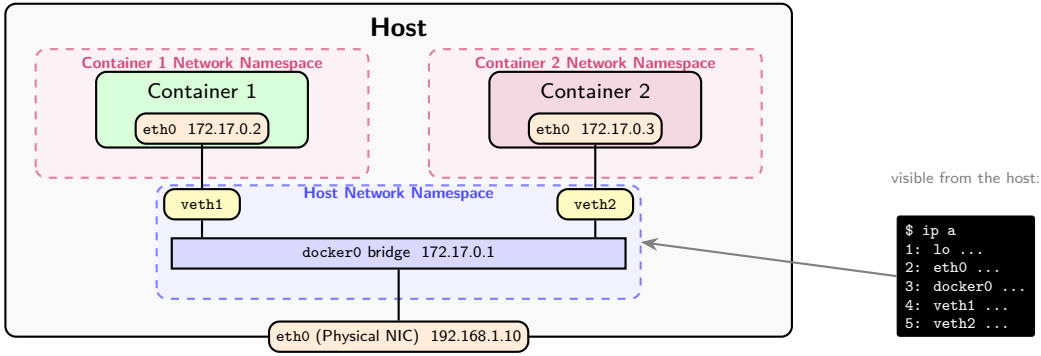
Each container needs its own network identity, must reach other containers (possibly on different hosts), and must be reachable from the outside. The overlay networks we just studied are exactly what makes this possible at scale.

How does networking work inside and between container hosts?

upf.

Container Networking: Starting Point

- Every container gets its own **network namespace**: isolated IP stack, interfaces, routing table
  - Network namespace: private isolated environment where the container sees only its own interfaces and routes
- Docker wires containers via **virtual bridges** and **veth pairs** (virtual cable)
- Containers reach each other, the host, and the internet depending on the **network model**

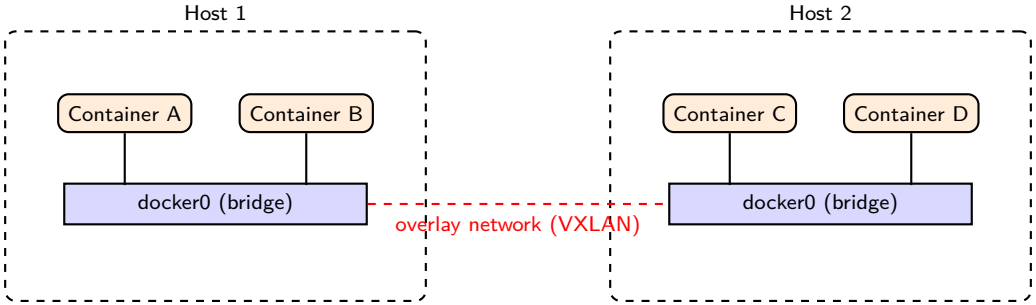


Container Network Models

- **Bridge network (default):**
  - Containers on the same host connected via a **virtual bridge** (`docker0`) [1]
  - Each container gets a **veth pair** (virtual Ethernet interface)
  - **NAT** for external traffic; containers share host's IP
- **Host network:**
  - Container uses the **host's network stack** directly
  - No network isolation, but **no NAT overhead**
  - Use for performance-critical applications
- **Overlay network:**
  - Containers on **different hosts** connected via VXLAN overlay
  - Same overlay concept we just covered, now applied to containers
  - Enables **multi-host** container deployments

[1] Docker, Inc., "Docker Networking Documentation," docs.docker.com/network.

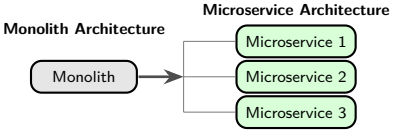
Container Networking: Visual

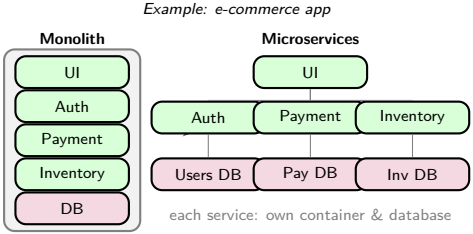


- Within a host: containers use a **bridge** network
- Across hosts: an **overlay** connects the bridges (VXLAN from the previous section)

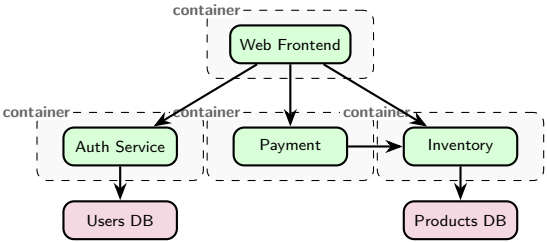
Microservices: From Monolith to Distributed

- **Monolithic** app: one big process, one deployment, one codebase
- **Microservices**: app split into small, independent services, each in its own container
- Rule of thumb: **1 microservice = 1 container**





Microservices: Network Challenges



- Services communicate over the **network** (HTTP/REST, gRPC)
- Network implications for hundreds of containers:
  - **Service discovery**: "where is the payment service?"
  - **Load balancing**: distribute requests across replicas
  - **Observability**: trace requests across services
- Every arrow = **network call** → latency and failure risk
- If Inventory is down → Payment and Web are affected (**cascading failure**)
- Solutions: retries, timeouts, circuit breakers, service meshes

upf.

Discussion: Containers vs VMs

If containers are faster and lighter than VMs,
why do companies still use VMs?

- Hint: security isolation, legacy applications, compliance requirements
- In practice, containers often run inside VMs

Outline

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. Cloud-Native Applications
5. From VLANs to Overlay Networks
6. Container Networking
7. **Infrastructure as Code & Network Automation**
8. Session Summary

upf.

Why Infrastructure as Code?

We can now program the network (SDN), virtualize its functions (NFV), scale tenant isolation (overlays), and run applications as containers following cloud-native principles. But if all of this is still configured by hand (clicking dashboards, running commands device by device) we are back to exactly the problem we started with: slow, error-prone, impossible to reproduce.

The final piece is treating infrastructure the same way we treat software: configuration written in files, reviewed in pull requests, stored in version control, and applied automatically.

That closes the loop from the manual networks we saw at the start of this session.

upf.

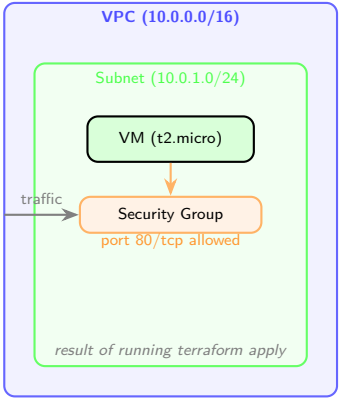
Infrastructure as Code (IaC)

- **IaC**: manage infrastructure through **code files**, not manual clicks
- Describe desired state in a configuration file → tools apply it automatically
- **Reproducibility**: same config → same infrastructure, every time
- **Version control**: track changes in Git, review, rollback
- **Automation**: no manual steps → fewer human errors
- **Speed**: deploy entire environments in minutes [1]

[1] Morris, *Infrastructure as Code*, 2nd ed., O'Reilly, 2021.

IaC Example: Terraform (Conceptual)

```
# Define a VPC [1]
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
# Define a subnet
resource "aws_subnet" "public" {
  vpc_id   = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
}
# Define a VM (EC2 instance)
resource "aws_instance" "web" {
  ami     = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
  subnet_id = aws_subnet.public.id
}
# Define a security group
resource "aws_security_group" "web" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 80
    to_port = 80
    protocol = "tcp"
  }
}
```



[1] HashiCorp, "Terraform Documentation," terraform.io/docs.

Discussion: IaC Risks

If infrastructure is defined as code,
what happens when someone pushes a bug?

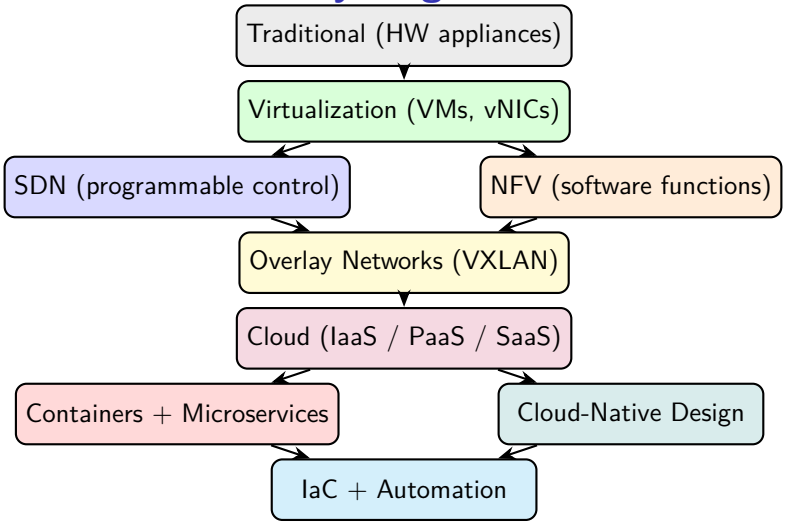
- Hint: code review, staging environments, automated testing
- How is this similar to software development best practices?

Outline

1. Limitations of Traditional Networks
2. Software-Defined Networking (SDN)
3. Network Function Virtualization (NFV)
4. Cloud-Native Applications
5. From VLANs to Overlay Networks
6. Container Networking
7. Infrastructure as Code & Network Automation
8. **Session Summary**

upf.

The Full Picture: Where Everything Fits



Each layer builds on the previous one. This is the **evolution of networking**.

Key Takeaways

1. Traditional networks are **manual, rigid, and do not scale**
2. **SDN** separates control from data plane: centralized, programmable, API-driven
3. **NFV** replaces hardware appliances with software: flexible, fast to deploy, cheap
4. SDN and NFV are **complementary**: SDN steers traffic, NFV processes it
5. **VXLAN** overlays scale isolation to 16 million networks (vs 4,096 VLANs)
6. Containers use **bridge networks** on a single host and **overlay networks** across hosts
7. **Cloud-native** means designed for the cloud: microservices, containers, declarative config, immutable infra, automated delivery
8. **Microservices** require service discovery, load balancing, and resilience mechanisms (circuit breakers)
9. **IaC** automates infrastructure: reproducible, version-controlled, auditable

upf.

Discussion

A company runs 200 microservices in containers.
One Friday at 5 PM, a manual network change
breaks communication for 50 of them.
How could SDN and IaC have prevented this?

Hints: centralized control, automated testing, instant rollback, reproducibility.

References

1. McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM CCR*, vol. 38, no. 2, 2008.
2. ONF, "Software-Defined Networking: The New Norm for Networks," ONF White Paper, 2012.
3. OpenDaylight Project, opendaylight.org.
4. ONOS Project, onosproject.org.
5. ETSI, "Network Functions Virtualisation," White Paper, 2012.
6. ETSI GS NFV 002, "NFV Architectural Framework," v1.2.1, 2014.
7. Docker, Inc., "Docker Networking Documentation," docs.docker.com/network.
8. Morris, *Infrastructure as Code*, 2nd ed., O'Reilly, 2021.
9. HashiCorp, "Terraform Documentation," terraform.io/docs.
10. Goransson & Black, *Software Defined Networks*, 2nd ed., Morgan Kaufmann, 2017.
11. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
12. IETF RFC 7348, "Virtual eXtensible Local Area Network (VXLAN)," 2014.
13. IETF RFC 8926, "Geneve: Generic Network Virtualization Encapsulation," 2020.
14. CNCF, "Cloud Native Definition v1.0," github.com/cncf/toc, 2018.
15. Burns et al., *Kubernetes: Up and Running*, 3rd ed., O'Reilly, 2022.

upf.